Deep Anomaly Detection for Tracking Experiments at LHCb

Carlo Peron
Department of Advanced Computing Sciences
Faculty of Science and Engineering
Maastricht University
Maastricht, The Netherlands

Abstract*—The task of particle tracking, or track reconstruction, has major importance in the field of high-energy physics. Charged particles are tracked using multiple detectors stacked in sequence. The following study will concentrate on the tracking experiments made at LHCb. The reconstruction of the particles' trajectories is subject to a degree of error. In fact, only a percentage of particles' trajectories, that are reconstructed, are correct. The remaining tracks are referred to as ghost tracks" or "clone tracks". A *ghost track* is a reconstructed track where a set percentage of hits are not from a real track. The research that will be proposed in this paper aims at reducing the number of ghost tracks from the ones that have been reconstructed across all the detectors. This task can be then characterized as an anomaly detection problem. The techniques that will be tested to address the issue are researched algorithms in the field of anomaly detection, namely "Autoencoders", "Variational Autoencoders", "Contractive AutoEncoders" and "Adversarially learned anomaly detection". Given the problem at hand, a machine-learning technique seems to be a promising route. One of the motivations lies in the fact that the process needs to be run on parallelized GPUs. The nature of such hardware allows for significantly faster computation in matrix operations, which are the backbone of deep learning and more machine learning methods. The goal of the study is to test the cited techniques and discover whether these achieve meaningful results. Whether the techniques improve on the previous results, these could then be implemented in more high-performance frameworks such as C++ and CUDA, to make the methods portable to the CERN system. [22]

I. Introduction

Housed at CERN, the European Organization for Nuclear Research, the LHC is the world's largest particle accelerator. In the accelerator, two beams of protons accelerated at nearly the speed of light they are made to collide. LHCb is a research experiment conducted at the LHC accelerator, dedicated to investigating the differences in matter and anti-matter by studying the subatomic particle quark-b. The goal of the LHCb experiment is therefore to perform precision measurements of b mesons. In doing so the primary objective of the LHCb experiment is to investigate the fundamental principle of charge conjugation parity (CP) violation, which involves exploring the underlying symmetry between matter and antimatter. This experiment seeks to understand the specific conditions and mechanisms that lead to the breaking of this symmetry. This challenge is addressed by examining the characteristics and dynamics of systems and particles composed of "heavy quarks." These quarks are abundantly generated through high-energy collisions that are made possible by the accelerator. [4]

It employs a set of subdetectors designed to detect the sub-particles and detect them before the decay. These are in order of proximity to the particles' beam: *VELO* (Vertex Locator), *RICH-1* (Ring-imaging Cherenkov detector), *Tracker Turicensis*, Outer Tracker, Inner Tracker, *RICH-2*, *ECAL* (Electromagnetic Calorimeter), *HCAL* (Hadronic Calorimeter) and finally the *Muon System*. Fig.1 below displays a longitudinal section of the sub-detectors. [2]

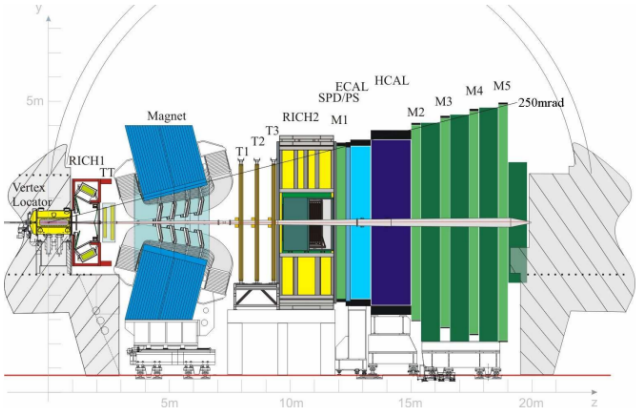


Figure 1. LHCb detector overview

The passage of the particles on the sensors of the sub-detectors is called "hit". Joining the hits of particles allows the researchers to sketch their trajectories, forming what are referred to as tracks. The tracks can be correct or instead, be mistakingly reconstructed by the algorithms. The tracks that contain two or more incorrect hits are then called ghost tracks. LHCb tracking algorithms are restricted in their performance by the number of *ghost tracks* Reducing ghost tracks can therefore facilitate further analysis of the data and in turn help in the understanding of these particles and their nature. Here we are investigating whether it is possible to reduce ghost tracks using deep learning methods. These techniques are tested against the data generated directly from the LHCb's tracking group.

This thesis was prepared in partial fulfilment of the requirements for the Degree of Bachelor of Science in Data Science and Artificial Intelligence, Maastricht University. Supervisor(s): [Daniel Hugo Cámpora Pérez, Kurt Driessens]

[0]Image Source: The LHCb detector at the LHC [2]

II. Related Work

In this context, anomaly detection is carried out by, the "Looking Forward" algorithm, which achieves about 20% on ghost particle detection. It works by setting a threshold for the *chi2* and by allowing for a minimum number of hits. Separately more techniques, based on the previous, have been researched. [19]

Recent studies indicate that machine learning techniques are becoming increasingly prevalent in high-energy physics [6], [13]. Considering the highly parallelized GPU architecture [1] used for tracking experiments and the growing literature on deep learning methods for anomaly detection [8], [21], adopting a deep learning solution is a promising approach for anomaly detection in tracking. Therefore, implementing a deep learning approach can potentially offer significant advantages in terms of accuracy and efficiency in tracking anomaly detection in this field. It's also been established how some of the researched techniques [7], [11], [16] have proven accuracy on similar data.

III. The Data

The data used for the study was directly generated from the simulator that is used to run the experiments of the *CERN* tracking group. The simulator called "Boole and Gauss" it used all across the *LHCb* research groups to simulate events in *LHCb*. It is Monte Carlo simulation. [10] The dataset is composed of 33545 observations. The data consisted of *JSON* objects, each one of which included the features:

- **isghost**: A Boolean value indicating whether the track is a ghost track or not.
- **x**: The x value at the track's state of z.
- **y**: The y value at the track's state of z.
- **z**: The state of the track.
- **tx**: The x-component of the track's direction vector at state z.
- **ty**: The y-component of the track's direction vector at state z.
- **qop**: The ratio of the charge to the momentum of the track at state z.
- **nb_hits**: Number of hits recorded in the track.
- **chi2**: χ^2 value for the the whole track.
- **trk_best_pt**: The track's best value the transverse momentum.
- **trk_best_qop**: The track's best recorded qop.
- **trk_chi2**: Tracker's χ^2 value.
- **trk_chi2T**: The track's χ^2 value at the *SciFi* subdetector.
- **trk_chi2V**: The track's χ^2 value at the *VELO* subdetector.
- **trk_first_qop**: The first measurement of the qop of the particle's track.
- **trk_kalman_docaz**: The z coordinate of from the distance of closest approach to the track, found by the *Kalman filter*.
- **trk_kalman_ip**: Impact parameter found by the *Kalman filter*.
- **trk_kalman_ip_chi2**: χ^2 value of the impact parameter found by the *Kalman filter*.
- **trk_kalman_ipx**: x coordinate of the impact parameter found by the *Kalman filter*.
- **trk_kalman_ipy**: y coordinate of the impact parameter found by the *Kalman filter*.
- **trk_ndof**: Number of hits in the tracker's segment.
- **trk_ndofT**: Number of hits in the tracker's segment at *SciFI* subdetector.
- **trk_ndofV**: Number of hits in the tracker's segment at *VELO* subdetector.
- **trk_velo_docaz**: The z coordinate of from the distance of closest approach to the track, computed by the *VELO* subdetector.
- **trk_velo_ip**:Impact parameter found by the *VELO* subdetector.
- **trk_velo_ipx**: x coordinate of the impact parameter found by the *VELO* subdetector.
- **trk_velo_ipy**: y coordinate of the impact parameter found by the *VELO* subdetector.
- **trk_velo_ip_chi2**: χ^2 value of the impact parameter found by the *VELO* subdetector.

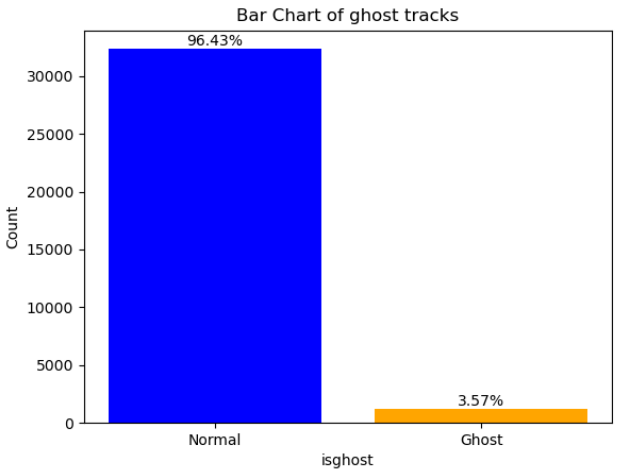


Figure 2. Distribution of the anomalies[1]

The *isghost* feature in the dataset exhibits an imbalanced distribution with 3.75% of anomalies and the remaining majority representing normal data. There is therefore a significant disparity between the two classes. Fig. 2 illustrates the distribution of the *isghost* feature, with anomalies represented by the minority class.

Prior to analysis, the dataset was converted to a more portable *CSV* file. The dataset has been divided into training and testing sets, with 80% of the data used for training and the remaining 20% used for testing. To make it so the neural network learned the normal distribution of the data, only normal instances were included in the training set. This choice is due to the fact that all the studied techniques use some sort of reconstruction error to distinguish between "normal" and anomalous" instances of the data. They, therefore, need to learn the normal distribution of the data to then evaluate if the analyzed instance falls into it.

Both the training and testing sets were then standardized using the mean and standard deviation of the training set.

This has been done considering that the testing set represents some real-world data. Taking the mean and variance of the whole dataset would have therefore introduced future information into the training variables. To make the features more comparable and improve the performance of some machine learning algorithms, they were scaled to a common range using a linear transformation. However, it is important to note that this technique, referred to as *normalizing*, may have impacted the relevance of extreme values, which should be considered when interpreting the results. Normalization is essential for neural networks as it expedites the learning process and becomes particularly crucial when the neural networks have to learn the distribution of the data over multiple variables. By ensuring that no feature scale dominates the others, normalization prevents biases and allows for fair and accurate comparisons.

IV. Methods

The following research aims at comparing by experiment, which of the techniques here presented is best suited for the task at hand. The techniques are gonna be compared on the performance regarding the anomaly detection task, the training time, and finally, the time it takes to classify an instance as anomalous or not. Also during the development, analysis, and testing of the algorithms, eventual flaws of the methods will be stated. This is to prevent an otherwise effective technique could be used out of context.

The methods will be tested on different hardware, and GPUs, to further eliminate uncertainty around the results. Each result will be reported with its confidence intervals and the statistical measures coming from the aggregated outcomes will also be taken in consideration before drawing any conclusion. Due to the extreme complexity of the system on which the track reconstruction is carried out, it will not be possible to test the algorithms on similar hardware. Only if this research yields any meaningful results is there a chance that the chosen algorithm could be tested on the native tracking hardware.

A. Autoencoders

Autoencoders are a class of neural networks, that are used for multiple purposes, such as image compression, image denoising, feature extraction, and dimensionality reduction. An *Autoencoder* works by encoding the input into a lower dimensional latent space and then decoding the latent representation back to the original dimension. It is composed of two networks in itself: an encoder and a decoder. The encoder is the one part that performs the dimensionality reduction. The decoder's objective is to reconstruct the input data. In between the two networks sits the bottleneck layer whose objective is to learn the latent representation of the data. [3] The *Stacked Autoencoders* are multiple encoders stacked on top of one another. These allow for more robustness and to have more trainable variables among weights and biases. In this study, every *Autoencoder* used is indeed a *Stacked Autoencoder*, having more than one hidden layer. Fig. 3 below displays the diagram of an *AutoEncoder*. Fig. 4 displays instead the architecture of a *Stacked AutoEncoder*.

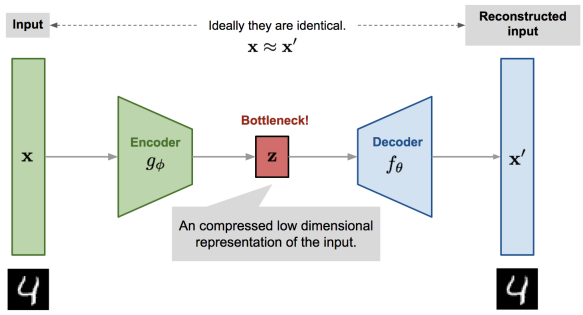


Figure 3. Autoencoder Architecture[2]

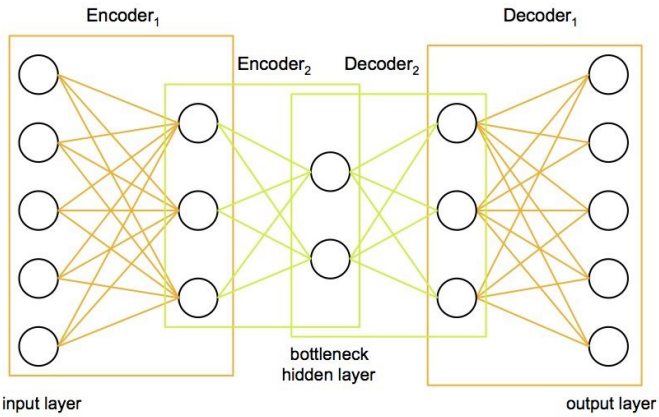


Figure 4. Stacked Autoencoder Architecture[3]

The use of these in anomaly detection is made possible by having them trained on "normal" data from the distribution. The "regularities" that will be learned by the network can then be used to detect those instances that are in fact not regular [9], [18], [21]. The *Autoencoder*, having learned the distribution of normal data, struggles to reconstruct anomalous data, and hence the reconstruction error is higher. Every instance that has a reconstruction error higher than a set threshold is then declared an anomaly.

Autoencoders have been used in multiple high-energy physics experiments, for track finding or anomaly detection. There is considerable literature about these in the context of anomaly detection. Plus because of the very nature of *Artificial neural networks* (ANN), the runtime can be easily cut down by introducing parallelization and *GPU* computing. In fact, the training and use of neural networks are mostly linear algebra operations, which have shown to be much faster on GPUs, using CUDA for example (up to factors of 20x). The downside of *AutoEncoders* is the fact that they need to be trained on "normal" data, not necessarily easy to collect, and moreover that they're sensitive to noise in the data. [5], [12], [20]

1) Contractive autoencoders: *Contractive Autoencoders* (CAEs) are a class of *Autoencoders* that learns a low-dimensional representation of the input data while minimizing the reconstruction error between the input and the output. In addition, these add a regularization term called the *contractive*

[2]Image source: AutoEncoder Architecture

[3]Source: Stacked AutoEncoder Architecture

loss to the loss function [24]. The contractive loss helps to constrain the mapping between the input and the encoded representation, which makes the learned features more robust to small variations in the input data. The loss term penalizes instances where a small change in the input leads to a large change in the encoding space. The contractive loss is defined as the *Frobenius norm* of the *Jacobian* matrix of the encoder with respect to the input. Mathematically, if $\mathbf{h} = f_{enc}(\mathbf{x})$ is the encoded representation of the input $\mathbf{x}$, then the contractive loss is given by:

$$\mathcal{L} = \mathcal{L}_{\text{reconstruction}} + \lambda \sum_{i,j} \left(\frac{\partial \mathbf{h}_i}{\partial \mathbf{x}_j} \right)^2 \tag{1}$$

- $\mathcal{L}_{\text{reconstruction}}$ represents the reconstruction loss between real and reconstructed instances.
- h_i is the hidden representation.
- x_j is the input feature.
- λ is a regularization parameter.

The contractive loss penalizes the *Jacobian* matrix of the encoder for mapping similar inputs to different encoded representations. In doing so, the model is encouraged to learn a more compact and robust representation of the input data.

2) Variational autoencoders: *Variational Autoencoders* (VAEs) contrarily to the previously covered *Autoencoders* classes learn a probabilistic representation of the input data. This is achieved by modeling the latent variables as a probability distribution [15]. The input is therefore not mapped to a vector of values but to a probability distribution. The goal is therefore to maximize the chance of seeing x for the distribution $p_\theta(x) = p(x|\theta)$. This is often chosen to be a Gaussian distribution $N(x|\mu, \sigma)$ The *VAE* consists of two components: an encoder and a decoder. The encoder maps the input data $\mathbf{x}$ to a mean vector $\boldsymbol{\mu}$ and a variance vector $\boldsymbol{\sigma}^2$ in the latent space. The decoder then maps the latent variables $\mathbf{z}$ to the output data $\mathbf{x}'$. Fig. 5 shows the architecture and the functioning of a *VAE*.

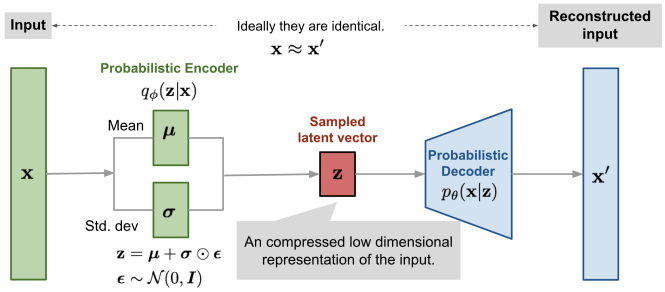


Figure 5. Variational Autoencoder Architecture[4]

The *Kullback–Leibler divergence* (KL divergence) between two probability distributions measures how much the two diverge from each other. Minimizing the KL divergence means optimizing the probability distribution parameters (μ and σ) to closely resemble that of the target distribution. The KL divergence term in *VAEs* is used to ensure that the learned latent variable distribution is close to a prior distribution. The encoder and decoder are trained jointly to minimize the reconstruction error and the KL divergence between the prior and the posterior distributions over the latent variables. The *VAE* loss function is given by:

$$L_{VAE} = \sum_{i=1}^{N} \mathcal{L}(\mathbf{x_i}, \mathbf{x_i}') + \beta \cdot \text{KL}(q_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x_i})||p(\mathbf{z})) \tag{2}$$

- $\mathcal{L}$ is the reconstruction loss.
- KL is the Kullback-Leibler divergence.
- β is a hyperparameter that controls the weight of the KL divergence term.
- $q_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x_i})$ represents the approximate posterior distribution over the latent variables given the input $\mathbf{x_i}$.
- $p(\mathbf{z})$ represents the prior distribution over the latent variables.

VAEs are well-suited for modeling the underlying distribution of the data, making them ideal for anomaly detection. This is due to the fact that this class of models is better able to generalize, and this also makes the model behave better on unseen data drawn from the same distribution.

B. ALAD: Adversarially learned anomaly detection

GANs (Generative Adversarial Networks) are a class of neural networks that are composed of two sub-networks: the generator *G* and the discriminator *D*. The generator aims to generate synthetic data $\widehat{x}$ that is similar to the real data *x*, while the discriminator aims to distinguish between the real and the generated data. *GANs* aim to generate new data that follows the same distribution as the training set by learning the underlying patterns and generating new instances from it [7]. The residual between a real instance and its reconstructed version can be used as the *anomaly score* for incoming instances. The underlying assumption is that normal data can be generated more easily than anomalies. However, training *GANs* can be challenging due to issues such as convergence problems or mode collapse, and they are more suitable for high-dimensional data.

A Bidirectional Generative Adversarial Network (BiGAN) extends a traditional Generative Adversarial Network (GAN) by incorporating an encoder network. The encoder learns to map the real data samples, to a latent space representation z. In a BiGAN, both the real space X and the latent space Z are used as input to the discriminator network. [14]

The *Adversarially Learned Anomaly Detection* (ALAD) model is a type of *BiGAN* that includes two additional discriminators. The extra *Discriminators* aid in the convergence of the model by enforcing *cycle-consistency* conditions. Therefore *ALAD* aims to improve the detection of anomalies by utilizing the information from both the real data space and the latent space [27]. Fig. 6 below displays the functioning of the *ALAD* method.

[4]Image source: Variational AutoEncoder Architecture

[5]Image source: ALAD Architecture

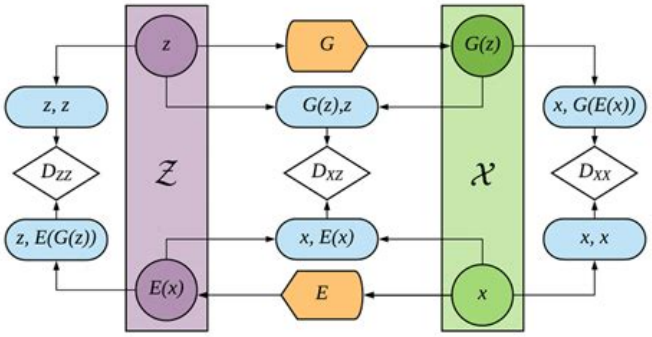


Figure 6. ALAD Architecture[5]

Adversarially learned anomaly detection (ALAD) is a method for detecting anomalies in data by training a *GAN* to distinguish between normal and anomalous data. The generator in the *GAN* is trained to generate synthetic normal data, while the discriminator is trained to distinguish between the synthetic normal data and the anomalous data. During the training process, the generator learns to generate synthetic data $\hat{x}$ that is similar to the normal data *x*, while the discriminator learns to become better at distinguishing between normal and anomalous data. The method is trained by solving the following value functions (by optimization).

The value function for the generator and the encoder is given here:

$$\min_{G,E}\max_{D_{xz}} \mathbb{V}(D_{xz},E,G) = \min_{G,E}\max_{D_{xz}} \mathbb{E}_{x\sim p_X}[\log D_{xz}(x,E(x))] + \mathbb{E}_{z\sim p_Z}[\log(1-D_{xz}(G(z),z))] \tag{3}$$

- $E(x)$: This symbolizes the encoder network, which takes real samples x as input and produces their corresponding encoded representations.
- $G(z)$: This represents the generator network, which generates fake samples using random latent variables z. Its aim is to produce samples that resemble real data.
- D_{xz}: This symbolizes the network that discriminates between *x* and z
- $\mathbb{E}x \sim p_X[\log Dxz(x,E(x))]$: The expected value of the discriminator's probability output for real samples x paired with their corresponding encoded representations $E(x)$.
- $\mathbb{E}z \sim p_Z[\log(1-Dxz(G(z),z))]$: The expected value of the discriminator's probability output for fake samples generated by the generator G using random latent variables z.

Additionally, by determining the value function for the remaining two discriminators we come to the complete value function to be optimized.

$$V(D_{xx},E,G) = \mathbb{E}_{x\sim p_X}[\log D_{xx}(x,x)] + \mathbb{E}_{x\sim p_X}[\log(1-D_{xx}(x,G(E(x))))] \tag{4}$$

- D_{xx}: This represents the discriminator network that takes two instances for real samples x paired with themselves. It measures the discriminator's ability to distinguish between two real samples.
- $\mathbb{E}x \sim p_X[\log Dxx(x,x)]$: The expected value of the discriminator's probability output for real samples x paired with themselves. It measures how well the discriminator can recognize real samples as authentic.
- $\mathbb{E}x \sim p_X[\log(1-Dxx(x,G(E(x))))]$: The expected value of the discriminator's probability output for real samples x compared to the generated samples $G(E(x))$. It captures the discriminator's capacity in distinguishing between real and generated samples.

$$V(D_{zz},E,G) = \mathbb{E}_{z\sim p_Z}[\log D_{zz}(z,z)] + \mathbb{E}_{z\sim p_Z}[\log(1-D_{zz}(z,E(G(z))))] \tag{5}$$

- D_{zz}: Indicates the discriminator network that takes two instances of latent variables, both denoted as z as input. It measures the discriminator's ability to distinguish between two identical latent variables.
- $\mathbb{E}z \sim p_Z[\log Dzz(z,z)]$: The expected value of the discriminator's probability output for real latent variables z paired with themselves. It measures how well the discriminator can recognize real latent variables as authentic.
- $\mathbb{E}z \sim p_Z[\log(1-Dzz(z,E(G(z))))]$: The expected value of the discriminator's probability output for real latent variables z compared to the latent variables generated by passing through the generator G and encoder E. It captures the discriminator's ability in distinguishing between real and generated latent variables.

In conclusion, ALAD's training objective can be expressed as:

$$\min_{G,E}\max_{D_{xz},D_{xx},D_{zz}} (V(D_{xz},E,G) + V(D_{xx},E,G) + V(D_{zz},E,G)) \tag{6}$$

To generate the anomaly scores, the first step is to encode the batch of data *x* into the latent representation *z* using the encoder *E*, then to generate a sample of synthetic data *x* from it with the generator *G*. Also, in the context of the D_{xx} network, *f_xx(x, x)* refers to the activations of the layer immediately preceding the logits, and it is calculated for the given pair of input samples.

$$A(x) = f_{xx}(x,x) - f_{xx}(x,G(E(x))) \tag{7}$$

The *anomaly score* will therefore be higher when the samples are more likely to be anomalous. The final task will be one of setting a threshold to classify the samples as anomalous or not, based on this score.

C. Transfer Learning and Ensemble Model

The utilization of the proposed *AutoEncoder* architectures not only provides an effective means of *anomaly detection* but also offers the opportunity for leveraging *transfer learning* via *weight initialization*. By initializing the weights of the more advanced models with the ones learned by the *AE*, we can effectively transfer the learned knowledge from the initial model to the subsequent models. This weight initialization technique ensures shorter training times and, potentially, improved

results as the initialized weights serve as a valuable starting point for subsequent model training. By porting the learned representations, the advanced architectures benefit from the foundational understanding captured by the initial model. Only a few additional *epochs* of training would then be needed to perfect the classifiers. Additionally, these pre-trained weights can be particularly beneficial for accelerating the training process of the *ALAD* (Adversarial Learned Anomaly Detection). [17]

By combining the outputs of multiple models with their diverse architectures, each initialized with the learned representations, an ensemble model can be constructed. This could be done in the form of *Voting Ensembles* or *Meta Ensembles (model stacking)*. A *voting ensemble* is a model that combines the independent predictions of more models into a singular one. The ensemble might then implement either *majority voting* or *weighted voting*. In the first case, the output of the ensemble will be the most predicted class by the submodels. In the case of *weighted voting* the final prediction is the result of a sum of the predictions of all the submodels, weighted by some chosen parameters (model accuracy could serve this purpose).

Another powerful technique for constructing ensemble models is *model stacking* or *meta ensembles*. Here, instead of directly combining the predictions, a meta-model, also known as a *blender* or *meta-learner*, is trained to make predictions based on the outputs of the individual models. The meta-model learns how to best combine the predictions of the base models, taking into account their strengths and weaknesses. This stacked approach enables the ensemble to capture more complex relationships and improve the overall predictive performance. [25]

V. Experiments

Each method has been tested for accuracy, precision, recall, and F1-score, and its ROC curve alongside the precision-recall curve has been drawn. The testing has been performed on a sample of the data containing both normal and anomalous data, with a split of 80% for training and 20% for testing. The precision-recall curve has been found to be more accurate for anomaly detection, as it is less sensitive to imbalanced datasets and provides a better representation of the model's performance on the minority class. Finally given the high computation power required to run the experiments, these have been run on a randomized sample of 2000 instances.

Given all the methods proposed aren't designed to output a binary value to classify anomalies or normal data, a function had to be designed to discriminate normal samples from anomalies. The *anomaly score* used for the Autoencoders is the reconstruction error, or the difference between the original sample and the reconstructed one. Conversely the *ALAD* (Adversarially Learned Anomaly Detection), the *anomaly score* is directly output by the method as a value between zero and one. Not assuming any domain knowledge, two techniques have been designed. The first approach has been to define the threshold for anomaly detection as a linear combination of the mean *anomaly scores* and their standard deviation. The second technique is instead a percentile-based method, for which the threshold value is set as the *nth* percentile of the *anomaly scores* distribution. The methods have been compared one against the other for each anomaly detection technique proposed.

Additionally, a subject of the experiments is the possibility of making use of transfer learning, in the form of weight transfer. The question posed is whether is possible to achieve statistically comparable accuracy between methods that have been trained extensively in isolation and those that have had their weights initialized with transfer learning. The base model used in this case is the *Autoencoder*, as it's considered the most portable among the ones studied. This has been trained for 50 epochs, and then its weights have been passed to the other methods, layer by layer for the *CAE* and the *VAE*. The *ALAD* method has just had its encoder initialized like this. These models have then refined with 5 epochs of training each. The results are once again analyzed with the two-sample z-score test, to determine if there's any statistical difference between the scores of methods trained in isolation and those of the methods where transfer learning was used.

To further tune these methods two hyperparameters also have been tested, namely the *learning rate* and the *encoding dimension*. For each of the two parameters, an array of values was considered, and the value selected was the one that maximized the average *recall* of the methods.

Finally, the developed models have been tested against some established techniques in the field of *anomaly detection*. The techniques chosen for this purpose are *Isolation forests* (IF), *One Class Support Vector Machines* (O-SVM), and *Density-Based Spatial Clustering of Applications with Noise* (DBSCAN). These techniques have been chosen because each of them presents a different and novel approach to anomaly detection when compared to the methods studied.

To further compare the models' performance, the time required to perform the classification task has also been recorded.

For each of the experiments listed the results have been collected after 30 full cycles of training. The results are determined by a two-sample right-tailed z-score test. The test is developed to evaluate whether or not to reject the null hypothesis.

$$\begin{aligned} H_0 &: \mu_1 = \mu_2 \\ H_1 &: \mu_1 > \mu_2 \end{aligned} \tag{8}$$

For each pair of variables to be tested the *z-score* is computed and whenever such score exceeds the critical value, determined for $\alpha = 0.05$, the null hypothesis H_0 is rejected. When instead the *z-score* is below the critical value the conclusion to draw is that there is not enough evidence to reject the null hypothesis H_0.

A. Experimental setup

The hardware used for the experiments was a Huawei Matebook laptop with processor 11th gen Intel(R) core i7-1165G7 @2.80GHz, with a Local physical memory (RAM) of 16 Gb.

The researched methods' neural network architectures are shown in Tables I, II. It is important to note that for a more fair comparison, all the *Autoencoders* have been trained using the same architecture shown below, but in the case of the *VAE* the encoder also employs a final custom layer. Is this custom

layer that performs the resampling of the latent space. It is also meaningful to note that at least in the case of *ALAD* the types of layers, their initialization, activation function, and dropout rate are directly obtained by the official implementation of the method.

Conversely, when developing the *Autoencoders* the choice of having a fixed dropout rate was to prevent overfitting [26]. While instead, the choice of using the *ReLu* activation function is due to the fact that this has proved to be one of the more versatile for Deep Neural Networks (DNN) [23].

Table I
AUTOENCODER ARCHITECTURE

Operator	Units	Activation	Dropout
Encoder			
Dense	512	ReLu	0.1
Dense	256	ReLu	0.1
Dense	128	ReLu	0.1
Dense	64	ReLu	0.1
Dense	32	ReLu	0.1
Dense	16	ReLu	0.1
Dense	3	ReLu	0
Decoder			
Dense	16	ReLu	0.1
Dense	32	ReLu	0.1
Dense	64	ReLu	0.1
Dense	128	ReLu	0.1
Dense	256	ReLu	0.1
Dense	512	ReLu	0.1

Table II
ALAD ARCHITECTURE

Operator	Units	Activation	Dropout
Encoder			
Dense	512	ReLu	0.1
Dense	256	ReLu	0.1
Dense	128	ReLu	0.1
Dense	64	ReLu	0.1
Dense	32	ReLu	0.1
Dense	16	ReLu	0.1
Dense	3	ReLu	0
Generator			
Dense	128	ReLu	0
Dense	128	ReLu	0
Dense	128	ReLu	0
Dense	31	ReLu	0
Discriminator$_{xx}$			
Dense	256	ReLu	0.1
Dense	512	ReLu	0.1
Dense	1024	ReLu	0.1
Output	1	Sigmoid	0
Discriminator$_{zz}$			
Dense	256	ReLu	0.1
Dense	512	ReLu	0.1
Dense	1024	ReLu	0.1
Output	1	Sigmoid	0
Discriminator$_{xz}$			
Dense	256	ReLu	0.1
Dense	512	ReLu	0.1
Dense	1024	ReLu	0.1
Output	1	Sigmoid	0

VI. RESULTS

The first experiment on the *thresholding* technique has shown that there isn't a significant difference between the two tested methods. It follows the percentile-based method and the one where a linear combination of the mean *anomaly scores* and their standard deviations are equivalent. There is no reason in fact to reject the null hypothesis H_0. Considering this result, from this point on only the "standard deviation" technique has been considered.

Similarly, the results of the experiment on transfer learning have shown that there is no statistically significant reason to reject the null hypothesis H_0 for the majority of the metrics tested. Hence it is possible to conclude that there is no significant difference in performance when transfer learning is used or not.

The conducted experiment pertaining to the hyperparameters, displayed in Table III, namely the *learning rate* and *encoding dimension*, facilitated the determination of optimal values for each method. These results are presented in Table IV. In subsequent experiments, the most frequently observed value for the *learning rate* in the results was chosen. A similar rationale was employed in determining the *encoding dimension*.

After comparing the results of the researched techniques with the ones of *IF* (Isolation forest), *O-SVM* (One-class Support Vector Machine) and *DBSCAN* (Density-Based Spatial Clustering of Applications with Noise), it has been shown that there exists no meaningful difference in performance between the first two and the models researched. It is instead possible that *DBSCAN* model performs significantly better than the models proposed. In fact the null hypothesis H_0 had been rejected for the majority of measures recorded.

As shown in Table IV the last experiment on the inference times of the models showed how the inference mechanism employed by *ALAD* is significantly faster than the other three. The *AutoEncoder* based techniques instead all showed similar results. This last result is likely due to the shared architecture of these models and their *reconstruction-based* anomaly detection.

As it shows there is no definitive "best" among the tested methods but each one proposes a different trade-off in the performance metrics. Even though the *CAE* method seems to perform always almost equally as well as the best, or better than the other two. The results to be considered in order to draw conclusions about the overall performance of the models are shown in Tables VI, VII, V.

DISCUSSION

The *Contractive Autoencoder* has achieved results comparable to the other methods if it didn't score the best. This alongside the fact it shares a similar architecture with the *AE* and *VAE* could open the door to the use of an *ensemble model*. The efficacy of *transfer learning* would make this possible by cutting down the time required for training all three models. The proposed approach would then become feasible due to the efficient utilization of a single model as a baseline for the other two, significantly reducing the training requirements. The meta-model would be able to leverage the intrinsic strengths of each technique and possibly give better results for the task at hand.

Nevertheless given the results achieved it is evident that none of the researched techniques, *CAE* included, appears to be a viable solution to this *anomaly detection* task. This can be shown by the fact that no technique would guarantee a

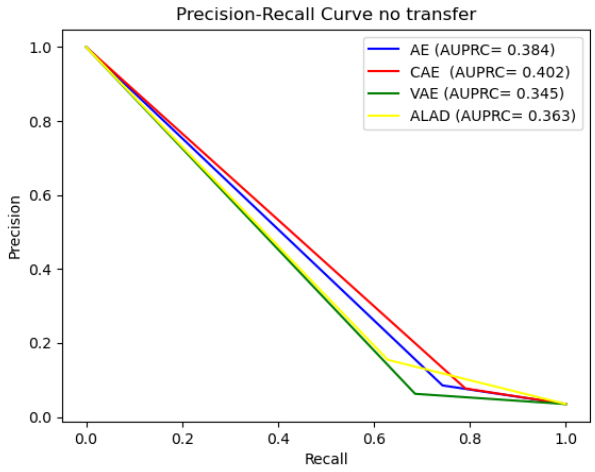


Figure 7. Precision Recall (AUC-PR)[6]

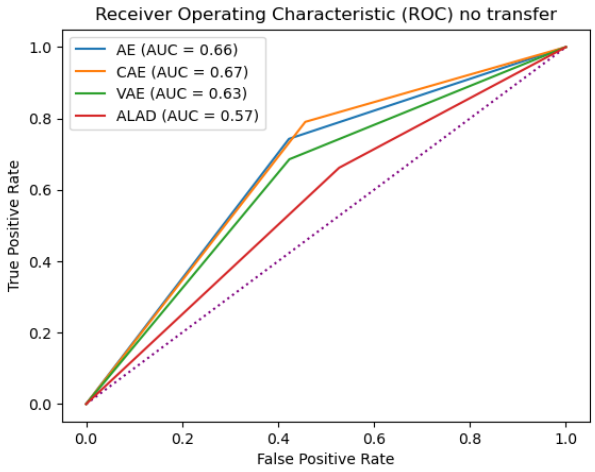


Figure 8. Receiver operating characteristic curve (AUROC)[7]

significant improvement in the reduction of *anomalies* in the face of its computation complexity. No method in fact proved to have high enough values for the *AUROC* or the *AUPRC*, as shown in Figure 8 and Figure 7. This is also to be considered true for *IF*, *O-SVM*, and *DBSCAN*.

VII. Conclusion

Given the unsatisfactory results of all researched methods, it seems possible that this class of methods doesn't hold discriminative power over the data. Keeping the dataset in this fixed form would imply that better techniques should be researched in *Statistical methods*, *Proximity-Based methods*, or *Rule-Based ones*. Alternatively, a reassessment of the data is warranted in order to guarantee enough predictive power.

Table III
Hyperparameter tuning

Method	Learning rate	Encoding dimension
AE	0.05	2
CAE	0.001	2
VAE	0.01	5
ALAD	0.003	3

Table IV
Mean inference times on 200 instances

Method	Inference time	Confidence Interval
AE	0.172	(0.113 - 0.23)
CAE	0.167	(0.145 - 0.188)
VAE	0.168	(0.161 - 0.174)
ALAD	**0.035**	(0.022 - 0.048)

Table V
Performance Evaluation

Method	Accuracy	Precision	Recall	F1-Score
AE	0.51	0.08	0.74	**0.13**
CAE	0.55	**0.08**	**0.79**	**0.13**
VAE	**0.58**	0.06	0.69	0.11
ALAD	0.51	**0.12**	0.63	0.10

Table VI
AUROC Comparison with Confidence Intervals

Method	AUROC	95% Confidence Interval
AE	0.66	(0.64 - 0.68)
CAE	**0.67**	(0.64 - 0.70)
VAE	0.63	(0.61 - 0.66)
ALAD	0.56	(0.54 - 0.59)

Table VII
AUPRC Comparison with Confidence Intervals

Method	AUPRC	95% Confidence Interval
AE	0.42	(0.38 - 0.45)
CAE	**0.44**	(0.41 - 0.47)
VAE	0.38	(0.35 - 0.41)
ALAD	0.40	(0.34 - 0.45)

References

[1] Roel Aaij, Johannes Albrecht, M Belous, P Billoir, T Boettcher, A Brea Rodríguez, D Vom Bruch, DH Cámpora Pérez, A Casais Vidal, DC Craik, et al. Allen: A high-level trigger on gpus for lhcb. *Computing and Software for big Science*, 4(1):1–11, 2020.

[2] A Augusto Alves Jr, LM Andrade Filho, AF Barbosa, I Bediaga, G Cernicchiaro, G Guerrer, HP Lima Jr, AA Machado, J Magnin, F Marujo, et al. The lhcb detector at the lhc. *Journal of instrumentation*, 3(08):S08005, 2008.

[3] Dor Bank, Noam Koenigstein, and Raja Giryes. Autoencoders. *arXiv preprint arXiv:2003.05991*, 2020.

[4] I Belyaev, G Carboni, N Harnew, C Matteuzzi, and F Teubert. The history of lhcb. *The European Physical Journal H*, 46(1):3, 2021.

[5] Andrea Borghesi, Andrea Bartolini, Michele Lombardi, Michela Milano, and Luca Benini. Anomaly detection using autoencoders in high performance computing systems. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 9428–9433, 2019.

[6] Dimitri Bourilkov. Machine and deep learning applications in particle physics. *International Journal of Modern Physics A*, 34(35):1930019, 2019.

[7] Olmo Cerri, Thong Q Nguyen, Maurizio Pierini, Maria Spiropulu, and Jean-Roch Vlimant. Variational autoencoders for new physics mining at the large hadron collider. *Journal of High Energy Physics*, 2019(5):1–29, 2019.

[8] Raghavendra Chalapathy and Sanjay Chawla. Deep learning for anomaly detection: A survey. *arXiv preprint arXiv:1901.03407*, 2019.

[9] Raghavendra Chalapathy and Sanjay Chawla. Deep learning for anomaly detection: A survey. *arXiv preprint arXiv:1901.03407*, 2019.

[10] M Clemencic, G Corti, S Easo, CR Jones, S Miglioranzi, M Pappagallo,

P Robbe, on behalf of the LHCb Collaboration, et al. The lhcb simulation application, gauss: design, evolution and experience. 331(3):032023, 2011.

[11] Marco Farina, Yuichiro Nakai, and David Shih. Searching for new physics with deep autoencoders. *Physical Review D*, 101(7):075021, 2020.

[12] Thorben Finke, Michael Krämer, Alessandro Morandini, Alexander Mück, and Ivan Oleksiyuk. Autoencoders for unsupervised anomaly detection in high energy physics. *Journal of High Energy Physics*, 2021(6):1–32, 2021.

[13] Katherine Fraser, Samuel Homiller, Rashmish K Mishra, Bryan Ostdiek, and Matthew D Schwartz. Challenges for unsupervised anomaly detection in particle physics. *Journal of High Energy Physics*, 2022(3):1–31, 2022.

[14] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks. *Communications of the ACM*, 63(11):139–144, 2020.

[15] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.

[16] Oliver Knapp, Olmo Cerri, Günther Dissertori, Thong Q Nguyen, Maurizio Pierini, and Jean-Roch Vlimant. Adversarially learned anomaly detection on cms open data: re-discovering the top quark. *The European Physical Journal Plus*, 136(2):236, 2021.

[17] Meenal V Narkhede, Prashant P Bartakke, and Mukul S Sutaone. A review on weight initialization strategies for neural networks. *Artificial intelligence review*, 55(1):291–322, 2022.

[18] Sheraz Naseer, Yasir Saleem, Shehzad Khalid, Muhammad Khawar Bashir, Jihun Han, Muhammad Munwar Iqbal, and Kijun Han. Enhanced network anomaly detection based on deep neural networks. *IEEE Access*, 6:48231–48246, 2018.

[19] M Needham. Identification of ghost tracks using a likelihood method. Technical report, 2008.

[20] Bryan Ostdiek. Deep set auto encoders for anomaly detection in particle physics. *SciPost Physics*, 12(1):045, 2022.

[21] Guansong Pang, Chunhua Shen, Longbing Cao, and Anton Van Den Hengel. Deep learning for anomaly detection: A review. *ACM Computing Surveys (CSUR)*, 54(2):1–38, 2021.

[22] Daniel Hugo Cámpora Pérez, Niko Neufeld, and Agustín Riscos Núñez. Search by triplet: An efficient local track reconstruction algorithm for parallel architectures. *Journal of Computational Science*, 54:101422, 2021.

[23] Prajit Ramachandran, Barret Zoph, and Quoc V Le. Searching for activation functions. *arXiv preprint arXiv:1710.05941*, 2017.

[24] Salah Rifai, Pascal Vincent, Xavier Muller, Xavier Glorot, and Yoshua Bengio. Contractive auto-encoders: Explicit invariance during feature extraction. pages 833–840, 2011.

[25] Omer Sagi and Lior Rokach. Ensemble learning: A survey. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 8(4):e1249, 2018.

[26] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958, 2014.

[27] Houssam Zenati, Manon Romain, Chuan-Sheng Foo, Bruno Lecouat, and Vijay Chandrasekhar. Adversarially learned anomaly detection. In *2018 IEEE International conference on data mining (ICDM)*, pages 727–736. IEEE, 2018.